

Enabling High-Goodput LLM Inference Serving with Heterogeneous SLO-Aware Speculative Decoding

Paper #1664

Abstract

Modern large language model serving must handle requests with heterogeneous *service-level objectives* (SLOs), making SLO-constrained goodput the primary optimization target. While speculative decoding offers per-request latency control, serial methods expose drafting overhead directly. Asynchronous parallel methods attempt to hide drafting latency but fail under batched execution, wasting massive compute on invalidated drafts and allocating capacity without SLO awareness, which starves urgent requests.

We present **SpecRhythm**, a speculative LLM serving system that logically decouples execution structure from speculation control to maximize goodput. SpecRhythm achieves this through three integrated techniques. First, a *Dual-Batch Execution Pipeline* organizes requests into two alternating batches, hiding drafting overhead within the complementary verification window to provide a stable, quantified drafting capacity. Within this window, *Rolling Eager Continuation* accelerates the most urgent requests by drafting ahead of schedule to meet their latency targets. Finally, *Individual Speculative Budget Shaping* dynamically allocates candidate tokens based on real-time SLO urgency and expected acceptance rates, directing limited speculative capacity toward requests with higher expected goodput benefit. Evaluations across diverse workloads show that SpecRhythm achieves up to $1.27\times$ higher goodput and $1.31\times$ higher SLO attainment compared to the best-performing baselines, highlighting its effectiveness in heterogeneous SLO serving.

1 Introduction

Transformer-based *large language models* (LLMs) [32, 33, 42] have achieved remarkable performance across a broad spectrum of tasks, such as interactive dialogue [23, 34], code completion [6, 22], document processing [2, 35], and complex reasoning [37, 38, 47]. LLM serving typically consists of two phases [1, 24]: the prefill phase, where the model processes all input tokens in the prompt (e.g., a request sequence) to generate the first output token, and the decoding phase, where the LLM generates one output token per iteration in an autoregressive manner.

In production LLM serving systems, users issue requests continuously [14, 40]. To overcome the low throughput of serial autoregressive decoding and ensure timely responses, continuous batching [10, 14, 40] is introduced to flexibly batch different user requests to generate multiple tokens in parallel within each decoding iteration. In practice, requests in the same batch often belong to diverse applications, which

exhibit varying *service-level objectives* (SLOs) [27, 28, 45], such as *time per output token* (TPOT) [11, 46], which is a critical metric to guarantee fast and consistent token generation performance. For example, interactive dialogue typically produces 10 tokens per second, which slightly exceeds human reading speed [3, 20], whereas code completion imposes a faster response speed (approximately 25 tokens per second) to maintain seamless interactions [29, 31]. As for document processing and complex reasoning tasks, they prioritize output quality over immediacy and can therefore tolerate a more relaxed TPOT target [21, 43].

Such heterogeneous TPOT SLOs pose significant challenges for mainstream LLM serving systems. As illustrated in Figure 1(a), existing systems like vLLM typically process batched requests with heterogeneous TPOT SLOs uniformly, leading to a mismatch between the identical per-token decoding latency and the multiple TPOT SLO requirements. Consequently, requests with tight TPOT constraints may suffer SLO violations, while requests with relaxed TPOT targets may consume valuable hardware resources that could otherwise be used to improve inference efficiency of stricter requests, ultimately undermining the ability to deliver reliable quality of service to users. This limitation has motivated SLO-constrained goodput optimization for LLM serving [17, 46]. Goodput is measured as the number of tokens generated per second for requests that successfully attain their SLO [43].

Recent studies [12, 17] suggest that speculative decoding [4, 5, 16, 19, 36] can improve goodput in heterogeneous-SLO serving. As illustrated in Figure 1(b), speculative decoding employs a lightweight draft model to predict multiple draft tokens for a given request and verifies them in parallel using the original target model. This allows each request in the same batch to yield a variable number of accepted tokens per decoding iteration, thereby enabling adaptive control of per-request TPOT without sacrificing throughput. Formally, a single request’s TPOT can be approximated as $\frac{T_{\text{draft}} + T_{\text{verify}}}{\mathbb{E}[A]}$, where T_{draft} and T_{verify} respectively denote the draft and verification latencies, and $\mathbb{E}[A]$ is the expected number of accepted tokens per decoding iteration. Given a batch of requests, let B denote the *total token budget*, defined as the total number of tokens verified in one decoding iteration. Let B_i denote the token budget assigned to the i -th request, i.e., the number of tokens from that request to be verified in the same decoding iteration. Note that the per-request budgets must satisfy the batch-level total token budget constraint, i.e., $\sum_{i=1}^{\text{BatchSize}} B_i \leq B$. Under this formulation, increasing B_i for a request with a tight TPOT

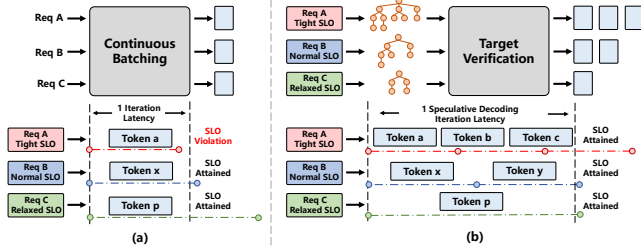


Figure 1. Motivation for SLO-aware speculative serving. (a) Continuous batching advances heterogeneous requests uniformly, while (b) speculative decoding exposes a tunable budget for different levels of request progress.

SLO can increase its expected number of accepted tokens, $\mathbb{E}[A]$, thereby reducing its TPOT and helping it meet the SLO requirement. Conversely, assigning a smaller budget to a request with a relaxed TPOT SLO can save compute resources while still preserving SLO attainment.

Existing SLO-customized speculative decoding systems [12, 17], have exploited adaptive token budget allocation to improve SLO attainment and goodput. However, they fall short of fully exploiting the optimization potential of speculative decoding. The root cause lies in the sequential execution of prediction and verification. Specifically, the target model must wait for the draft model to complete its prediction before starting verification, and the draft model must stall until verification finishes before proceeding to the next iteration. Within a batch, requests with smaller token budgets are forced to wait for those with larger ones to finish prediction before triggering verification. This synchronization increases the T_{draft} of requests with smaller token budgets. Worse still, the reduced budgets may also lower their $\mathbb{E}[A]$. The combination of longer draft latency and fewer accepted tokens exacerbates TPOT and may lead to SLO violations even for requests with relaxed TPOT requirements.

Although emerging asynchronous parallel speculative decoding technologies [18, 44] can hide draft latency by overlapping drafting with verification, it is insufficient to address the aforementioned issues due to two reasons. First, existing asynchronous designs often continue drafting during verification under the optimistic assumption that all previously drafted tokens can be accepted. When this assumption fails, the newly generated draft tokens become invalid and must be discarded, forcing execution back to the sequential draft-then-verify pattern. Such failures are common in practice because token acceptance is inherently uncertain. Second, existing studies primarily focus on maximizing the overlap between prediction and verification, while treating batched requests homogeneously without considering their specific SLOs. Consequently, visible draft latency reduction does not necessarily translate into goodput improvement.

In essence, conventional speculative decoding enables adaptive control over the number of accepted tokens per request in each decoding iteration, but increases the draft latency of batched requests. Asynchronous parallel speculative decoding mitigates this issue by overlapping prediction with verification, but the resulting hidden drafting capacity is fragile under prefix invalidation and is not allocated according to per-request SLOs. These limitations suggest that speculative decoding should not be treated merely as a request-level latency-hiding policy. Instead, it should be organized as a schedulable batch-level resource for heterogeneous-SLO serving.

In this paper, we present **SpecRhythm**, a speculative LLM serving system that maximizes goodput under heterogeneous TPOT SLOs. The key idea of SpecRhythm is to decouple the execution structure that creates drafting opportunities from the control policy that allocates them. The execution layer builds a stable batch-level overlap structure and exposes the resulting drafting windows as schedulable resources. The control layer then decides which requests should draft ahead of their normal batch turn and how much token budget each request should receive. This design turns speculative parallelism from a local latency-hiding technique into a batch-level resource allocation mechanism for SLO-constrained goodput optimization.

SpecRhythm realizes this idea through three techniques. First, the *Dual-Batch Execution Pipeline* organizes active decoding requests into two alternating batches. By placing the drafting stage of one batch within the verification window of the other, it hides draft latency while exposing a predictable drafting window for scheduling. Building on this pipeline, *Rolling Eager Continuation* lets selected urgent requests draft ahead of their regular turn using the available overlap window. If the target model fully accepts their current drafts, the pre-generated draft tokens can be immediately used in the next verification step, avoiding an additional drafting wait. If verification rejects any current draft token, the pre-generated draft tokens are discarded and the request falls back to normal drafting from the verified prefix, preserving generation correctness. Finally, *Individual Speculative Budget Shaping* assigns speculative budgets to individual requests under a roofline-guided verification constraint. It first allocates candidate tokens to close projected progress gaps for urgent requests, and then distributes the remaining capacity according to draft confidence and residual urgency to improve overall goodput.

In summary, this paper makes the following contributions:

- We formulate heterogeneous TPOT SLO speculative serving as a resource allocation problem under execution constraints, showing that high goodput requires both hidden drafting capacity and SLO-aware allocation of that capacity.

- We propose a dual-batch execution pipeline that overlaps drafting and verification across two logical batches, reducing exposed drafting latency and creating a predictable drafting window.
- We introduce rolling eager continuation and individual speculative budget shaping, which allocate hidden drafting capacity to urgent requests using runtime progress gaps and draft confidence.
- We implement SpecRhythm and evaluate it on NVIDIA A100 GPUs under heterogeneous workloads, achieving up to $1.27\times$ higher goodput and $1.31\times$ higher SLO attainment over state-of-the-art baselines.

2 Background and Motivation

2.1 Heterogeneous SLO Serving and Goodput

In streaming LLM serving, responsiveness after the first token is primarily captured by time per output token (TPOT), which measures the sustained generation rate during decoding. For a request r_i , let its target TPOT be τ_i , meaning that the request should be advanced with an average per-token decoding time no higher than τ_i . A smaller τ_i corresponds to a stricter latency target, while a larger τ_i indicates a more relaxed generation pace. Suppose r_i produces N_i output tokens, and let L_i denote the total decoding latency for producing these tokens. Its actual TPOT is defined as $\text{TPOT}_i^{\text{actual}} = \frac{L_i}{N_i}$. Request r_i satisfies its TPOT SLO if and only if $\text{TPOT}_i^{\text{actual}} \leq \tau_i$. This request-level definition captures the average sustained generation rate over the decoding phase, rather than enforcing a strict instantaneous interval between every adjacent token pair. It also allows requests with different output lengths, application scenarios, and SLO targets to be compared under a unified per-token serving metric.

Based on this definition, we use *goodput* as the primary optimization objective. Let $\mathcal{R}$ denote the set of requests completed or tracked within a measurement window of length T . Goodput counts the effective output tokens produced by SLO-satisfied requests, defined as

$$\text{Goodput} = \frac{1}{T} \sum_{r_i \in \mathcal{R}} N_i \cdot \mathbb{I}[\text{TPOT}_i^{\text{actual}} \leq \tau_i]$$

where $\mathbb{I}[\cdot]$ is the indicator function. Tokens produced by a request are counted toward goodput only if the request satisfies its TPOT SLO; otherwise, they are considered ineffective for SLO-constrained serving. Unlike throughput, which counts all generated tokens regardless of latency violations, goodput measures the effective serving capacity under heterogeneous SLOs. Alongside goodput, we report *SLO attainment*, calculated as

$$\text{Attainment} = \frac{1}{|\mathcal{R}|} \sum_{r_i \in \mathcal{R}} \mathbb{I}[\text{TPOT}_i^{\text{actual}} \leq \tau_i]$$

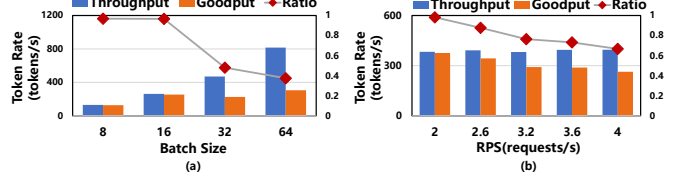


Figure 2. Throughput can diverge from goodput. Increasing (a) batch size or (b) request rate keeps throughput high but reduces the fraction of SLO-satisfied output.

Goodput is token-weighted, while SLO attainment is request-weighted. Together, they capture both the amount of useful output produced and the fraction of requests that meet their latency targets.

This distinction is important because high throughput does not necessarily imply high goodput. Prior goodput-oriented serving systems have shown that generated tokens lose service value when their requests miss SLO targets [43, 46]. Figure 2 illustrates the same effect in our setting: increasing batch size or request rate can keep throughput high, while the fraction of tokens generated by SLO-satisfied requests drops sharply. Therefore, optimizing heterogeneous SLO serving requires explicit goodput-oriented control rather than throughput maximization alone.

2.2 Speculative Decoding Basics

Speculative decoding accelerates autoregressive generation through a two-stage process of drafting and verification. A lightweight draft model proposes a sequence or tree of candidate tokens from the verified prefix, and the target model verifies these candidates in one iteration, accepting the longest prefix consistent with its distribution [5]. When drafts are accepted, one target-model iteration can commit multiple output tokens; when all drafts are rejected, the target model still returns one corrected token.

This process exposes a speculative budget for controlling the progress of requests. The budget denotes the amount of candidate token work assigned to a request in a single iteration, such as draft length or tree size. A larger budget can increase the expected number of accepted tokens $\mathbb{E}[A_i]$, but it also increases drafting work and may increase verification latency. This tradeoff makes speculative decoding attractive for heterogeneous SLO serving, because a tight-SLO request can receive more speculative opportunities while a relaxed request can conserve computation.

However, this budget-progress tradeoff becomes a batch-level control problem under heterogeneous workloads. Serial speculative serving [12, 17] incurs additional drafting and verification costs directly within the batch iteration, while asynchronous parallel speculation [18, 44] can hide draft latency but may waste hidden work when predicted prefixes

are rejected. These limitations motivate a closer examination of speculative serving under batched heterogeneous workloads.

2.3 Motivating Findings and Design Insights

Although speculative decoding can generate multiple tokens per verification, this capability does not automatically translate into high goodput. We conduct controlled experiments on a single node with 4 NVIDIA A100 GPUs, using Qwen3-32B as the target model and Qwen3-0.6B as the draft model. The workload contains three SLO classes with tight, normal, and relaxed TPOT targets. These experiments reveal two critical limitations in representative state-of-the-art speculative serving systems—namely, AdaServe, a serial system, and PEARL, an asynchronous parallel system.

Finding 1: Serial speculative decoding faces a latency-progress dilemma. Figure 3(a) shows that AdaServe increases throughput as batch size grows, but its goodput grows more slowly and the goodput-to-throughput ratio drops. Figure 3(b) explains this gap. To bound iteration latency, AdaServe limits the number of draft iterations and keeps the total verification budget per batch fixed. This prevents draft and verification latency from growing dramatically with batch size. However, it also means that more requests share the batch-level speculative capacity. As batch size increases, each request receives a smaller effective speculative budget, so the average number of accepted tokens per request decreases in each iteration.

This exposes the fundamental dilemma of AdaServe under batched serving. If the system fixes the total batch budget to bound iteration latency, per request token progress decreases as the batch grows. If it instead keeps the speculative budget of each request fixed, the total number of draft and verification candidates grows with batch size, causing the serial iteration latency to increase substantially. In both cases, serial execution couples token progress with iteration latency: improving one tends to hurt the other.

Insight. A goodput-oriented speculative serving system should avoid relying solely on serial budget scaling. It needs a batch-level execution structure that hides draft overhead while keeping verification cost bounded, so that speculative capacity can improve timely per-request progress.

Finding 2: Asynchronous parallel speculative decoding wastes hidden drafting capacity and lacks SLO-aware progress allocation. Figure 4(a) shows that PEARL improves throughput as batch size increases and slightly outperforms AdaServe in throughput. However, this throughput gain does not consistently translate into goodput. The goodput-to-throughput ratio decreases with larger batches, and the goodput at batch size 64 even drops below that at batch size 32. The reason is that hiding draft latency does not guarantee useful progress. Tokens drafted ahead are valid only if the current draft prefix is fully accepted. Under

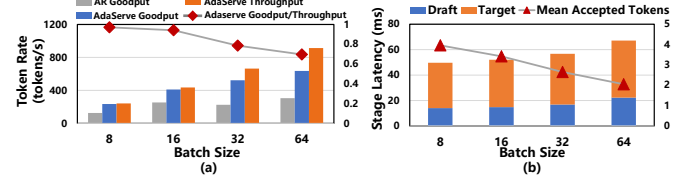


Figure 3. Latency-progress dilemma in AdaServe. (a) Throughput increases with batch size, but goodput grows more slowly. (b) The mean number of accepted tokens per iteration decreases as the batch-level speculative budget is shared across more requests.

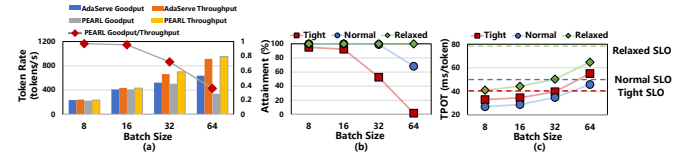


Figure 4. Limitations of PEARL. (a) Higher throughput does not consistently translate into higher goodput. (b) SLO attainment becomes uneven across request classes. (c) Tight requests exceed their TPOT target under larger batches.

PEARL, 38.4% of drafted tokens are discarded before verification because their prefixes are invalidated, and another 29.2% are rejected during verification. These tokens consume drafting or verification capacity without contributing to progress.

Figures 4(b) and 4(c) further show that this ineffective use of hidden drafting capacity is poorly aligned with heterogeneous SLO urgency. As batch size increases, tight request attainment drops sharply, while relaxed requests remain mostly satisfied. The TPOT measurements explain this imbalance: tight requests experience increasing TPOT and eventually exceed their corresponding SLO threshold, whereas relaxed requests stay below their looser target. Normal requests remain within the target on average, but a nontrivial fraction still violate the SLO, as reflected in their reduced attainment. This indicates that PEARL does not allocate the hidden drafting window to the requests that most critically need timely progress.

Insight. Hidden drafting capacity should be treated as a shared resource. To improve goodput, the system should allocate this capacity to requests that both need progress urgently and are likely to convert speculative work into accepted tokens.

3 Analysis and Problem Formulation

The findings in §2.3 motivate viewing speculative serving as an online resource allocation problem under execution constraints. At each service cycle, the scheduler must decide which requests receive speculative work, how much budget each request obtains, and whether the resulting draft and

verification workload can fit within the available execution window.

This section formalizes batched speculative serving under heterogeneous TPOT SLOs. The goal is not to derive an exact global optimum. Instead, the formulation identifies the control principles required for goodput-oriented speculation: drafting overhead should be hidden whenever possible, verification cost should remain bounded, and speculative capacity should be directed to requests that both need timely progress and are likely to convert draft work into verified output progress.

3.1 Runtime Request State

The goodput definition in §2.1 evaluates a request after completion or within a measurement window. Online scheduling, however, requires runtime estimates of whether each active request is falling behind its TPOT target. Let $\mathcal{R}_t$ denote the set of active decoding requests at the beginning of service cycle t . For each request $r_i \in \mathcal{R}_t$, the system tracks its delivered output tokens $N_i(t)$, accumulated decoding time $L_i(t)$, and TPOT target τ_i . Let $P_i(t)$ denote the verified output progress committed for request r_i in cycle t . In speculative decoding, $P_i(t)$ includes accepted draft tokens and any corrected target token returned after a rejection. The delivered token count is updated as $N_i(t+1) = N_i(t) + P_i(t)$. If the request is not selected for verified progress in cycle t , then $P_i(t) = 0$. In both cases, $L_i(t)$ increases by the cycle duration while the request remains active, capturing scheduling delay and making stalled requests increasingly urgent. We estimate the runtime TPOT of request r_i as $\widehat{\text{TPOT}}_i(t) = \frac{L_i(t)}{\max(N_i(t), 1)}$, where the denominator avoids division by zero and treats token-starved requests as urgent. Its normalized slack is $s_i(t) = 1 - \frac{\widehat{\text{TPOT}}_i(t)}{\tau_i}$. A positive slack indicates the request remains within its target, while a slack close to zero or below indicates it is near or beyond the violation threshold. This runtime state provides the basis for urgency-aware scheduling and speculative budget allocation.

3.2 Execution Constraints under Overlap

A service cycle consists of drafting and verification. Let $\mathcal{S}_t \subseteq \mathcal{R}_t$ be the set of requests selected in cycle t , and let $n_i(t)$ denote the speculative budget assigned to request $r_i \in \mathcal{S}_t$. The budget vector is $\mathbf{n}_t = \{n_i(t) \mid r_i \in \mathcal{S}_t\}$. Given the selected requests and their budgets, the batch drafting cost and verification cost are denoted as $D_t = C_{\text{draft}}(\mathcal{S}_t, \mathbf{n}_t)$ and $V_t = C_{\text{verify}}(\mathcal{S}_t, \mathbf{n}_t)$. Under serial speculative decoding, drafting and verification are executed sequentially, yielding $T_t^{\text{serial}} = D_t + V_t$. Thus, increasing speculative budgets directly increases iteration time whenever they increase drafting or verification work. This captures the latency progress dilemma observed in §2.3. Under overlapped execution, drafting can be placed into a hidden window provided by concurrent verification. Let W_t denote the available drafting

window in cycle t . The exposed drafting overhead is $\Delta_t = \max(0, D_t - W_t)$, and the cycle duration becomes $T_t = V_t + \Delta_t$. This model captures two constraints. First, drafting work improves latency only when it fits within the available window. Second, verification cost always contributes to the cycle duration, so the total verified candidate budget must remain bounded. Therefore, a goodput-oriented system must control both exposed drafting overhead and verification workload.

3.3 Scheduling and Speculative Budget Decisions

Under these execution constraints, the system controls two coupled decisions in each cycle. The first decision is request scheduling: selecting the subset $\mathcal{S}_t \subseteq \mathcal{R}_t$ that will receive speculative work. The selected set is constrained by the maximum active decoding concurrency, denoted as $|\mathcal{S}_t| \leq B_t$. The second decision is budget allocation: assigning each selected request a non-negative integer budget $n_i(t)$, where $n_i(t)$ denotes the number of candidate tokens assigned to request r_i in cycle t . We model the verified progress of request r_i in cycle t as a random variable $P_i(t)$, with expected value $g_i(n_i(t), t) = \mathbb{E}[P_i(t) \mid n_i(t), \mathcal{H}_i(t)]$, where $\mathcal{H}_i(t)$ summarizes the request's current prefix, recent acceptance behavior, and draft confidence. While larger budgets might increase the expected progress, the marginal benefit typically diminishes. Because speculative decoding requires strict prefix matching, a single rejected token immediately invalidates all subsequent drafts in that branch, causing the probability of continuous alignment to decay rapidly with depth. Furthermore, when using draft trees, the target model ultimately accepts only a single matching branch, so expanding the candidate width incurs wasted computation on all unselected branches. The two decisions are tightly coupled. $\mathcal{S}_t$ determines which requests can make progress, while $\mathbf{n}_t$ determines how much progress they may make and how much draft and verification costs they incur. Maximizing goodput, therefore, requires joint control over scheduling and speculative budgets under the shared execution constraints.

3.4 Utility for Goodput-Oriented Speculation

The preceding subsections define the runtime state, execution limits, and controllable scheduling decisions. We now use them to compare feasible request-budget assignments. A useful assignment should satisfy two conditions: the request should be close to violating its TPOT target, and the assigned budget should have high expected verified progress. We therefore define the utility of assigning budget $n_i(t)$ to request r_i as $U_i(n_i(t), t) = u_i(t) \cdot g_i(n_i(t), t)$, where $u_i(t)$ is an urgency score derived from the runtime slack $s_i(t)$ and $g_i(n_i(t), t)$ is the expected verified progress defined above. In our implementation, a smaller slack corresponds to a larger urgency, so requests near or beyond their TPOT boundary receive higher priority. This utility is not intended to solve a global offline optimization problem. Instead, it provides a per-cycle ranking principle for selecting among the feasible

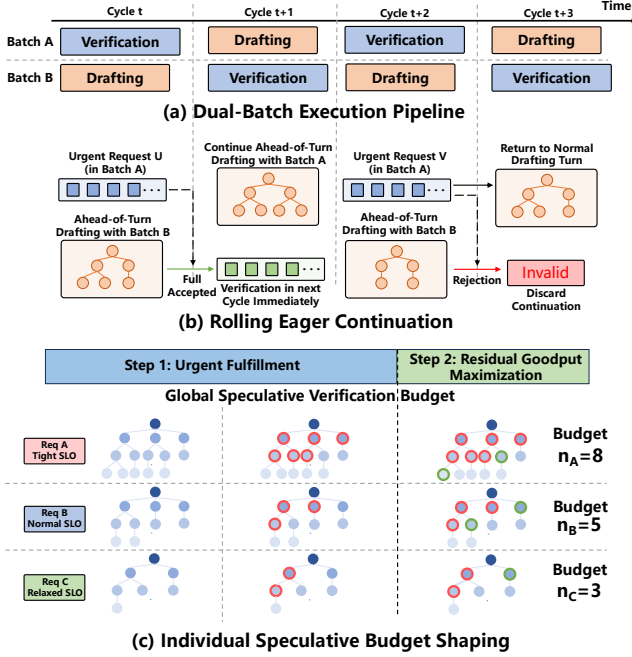


Figure 5. Overview of SpecRhythm. SpecRhythm combines dual-batch execution, rolling eager continuation, and individual speculative budget shaping to allocate speculative work across heterogeneous SLOs.

scheduling and budget choices constrained by S_t , n_t , Δ_t , and V_t . Assignments with high utility are preferred because they are expected to convert scarce speculative capacity into timely verified progress. Assignments with low utility are deprioritized either because the request still has sufficient slack or because additional candidate tokens are unlikely to be accepted. This formulation identifies the core control principle: speculative capacity should be assigned where it has high urgency-weighted expected progress while respecting batch-level execution limits.

4 Design

4.1 Overview

Building on the formulation in §3, SpecRhythm first hides drafting latency via a stable batch-level overlap structure, then allocates the resulting hidden drafting capacity based on runtime SLO urgency and expected verified progress. As illustrated in Figure 5, SpecRhythm consists of three integrated techniques. The **Dual-Batch Execution Pipeline** (§4.2) alternates two logical batches so that drafting for one batch runs inside the verification window of the other, reducing exposed drafting latency and creating a predictable drafting window. **Rolling Eager Continuation** (§4.3) uses this window to let urgent requests draft ahead of their normal batch turn. **Individual Speculative Budget Shaping** (§4.4) decides how many candidate tokens each selected request

receives based on projected progress gaps, draft confidence, and bounded verification cost. Together, these techniques separate the creation of hidden drafting capacity from its allocation, turning speculative parallelism into a serving-level control mechanism for SLO-constrained goodput.

4.2 Dual-Batch Execution Pipeline

SpecRhythm hides drafting overhead by dividing active requests into two logical batches and processing them through an alternating draft-verification pipeline. In each service step, the target model verifies the draft tokens of one batch, while the draft model generates new draft tokens for the other batch. In the subsequent step, the two batches swap roles. This structure decouples the two stages at the batch level: rather than requiring a single batch to complete drafting before verification and then wait for the next drafting stage, SpecRhythm schedules the drafting work of one batch during the verification window of the other. As a result, the system reduces exposed drafting latency and creates a regular overlap window for scheduling control. The scheduler maintains the two logical batches across service steps to keep the overlap window stable.

SpecRhythm realizes this overlap by separating draft and target execution resources when hardware permits. The draft model is lightweight relative to the target model, so it can often run on a smaller resource partition while the target model uses the remaining resources for verification. This separation reduces contention between drafting and verification and allows the draft worker to prepare draft tokens concurrently with target-model execution. The drafting workload of one batch is therefore placed into the verification window W_t provided by the concurrent batch. Following the model in §3, the overlapped cycle time can be approximated as $T_t^{\text{overlap}} \approx V_t + \Delta_t$, where Δ_t denotes the drafting overhead that remains exposed after overlap. When $D_t \leq W_t$, the drafting workload does not add delay beyond the verification time. When $D_t > W_t$, only the overflow contributes to the cycle time. In both cases, the verification cost V_t remains on the batch execution path, so speculative budgets must still be constrained to avoid increasing verification latency.

The primary value of this execution pipeline is not merely reducing drafting overhead. It also turns drafting capacity into a stable batch-level resource. By organizing drafting around the verification window W_t , SpecRhythm provides the control layer with a predictable bound on how much drafting work can be attempted without significantly perturbing the verification-dominated execution time. This regularity is the foundation for Rolling Eager Continuation and Individual Speculative Budget Shaping, which use the available window to decide which urgent requests should be drafted ahead and how much speculative budget each request should receive.

4.3 Rolling Eager Continuation

The Dual-Batch Execution Pipeline provides a regular overlap structure, but requests still advance according to the normal batch rotation. A request can prepare its next draft only when its batch reaches the drafting stage. This baseline schedule is sufficient for requests with relaxed TPOT targets, but it lacks the required scheduling agility for requests close to their SLO boundary. Such requests may continue accumulating latency while waiting for their next normal drafting stage, even when the system has residual drafting capacity during the current overlap window.

SpecRhythm introduces Rolling Eager Continuation to advance the execution of urgent requests. While the target model verifies the current batch, the draft model inserts selected urgent requests into the existing drafting window and generates subsequent tokens extending their unverified sequences. This mechanism does not create an additional execution batch. If the target model fully accepts the current draft, the pre-generated draft tokens are immediately promoted to a ready proposal, allowing the request to avoid waiting for the normal drafting stage. Conversely, if verification rejects the current draft, the pre-generated tokens are discarded, and the request falls back to normal drafting from the verified tokens.

To decide which requests should receive eager continuation, SpecRhythm estimates the projected progress gap each request may accumulate before its next normal execution turn. Let $\hat{T}_i^{\text{next}}(t)$ denote the estimated waiting time until request r_i can next obtain verified progress. The minimum number of verified output tokens needed to keep the request on track is defined as $a_i^{\text{need}}(t) = \left\lceil \left[\frac{L_i(t) + \hat{T}_i^{\text{next}}(t)}{\tau_i} - N_i(t) \right]_+ \right\rceil$, where $[x]_+ = \max(x, 0)$. A request with $a_i^{\text{need}}(t) = 0$ has sufficient slack, while a request with $a_i^{\text{need}}(t) > 0$ is projected to need additional progress before its next normal turn.

A projected progress gap alone is not sufficient for eager admission. If a request has recently experienced high rejection rates, generating continuations is likely to produce invalid computation. SpecRhythm therefore calculates an expected acceptance benefit $\hat{b}_i(t)$ for each request, derived from its recent acceptance behavior and the draft model's confidence. In our implementation, $\hat{b}_i(t)$ is estimated by multiplying the recent acceptance ratio of request r_i with the average draft-model confidence of the proposed continuation tokens. To determine scheduling priority, the system combines the progress gap with this expected benefit to compute an eager priority score $\pi_i(t) = a_i^{\text{need}}(t) \cdot \hat{b}_i(t)$. Instead of enforcing a fixed quota, SpecRhythm dynamically adapts the size of the eager execution set to available resources. The system estimates a provisional minimum speculative budget for each request to cover its projected gap. Requests are then ranked and admitted in descending order of priority, provided that their combined speculative budgets fit within the

remaining verification budget and their total drafting workload does not exceed the residual drafting window W_t . This formulation ensures that eager continuation is used only for urgent requests with a high expected benefit of acceptance.

4.4 Individual Speculative Budget Shaping

With both normal drafting and eager continuation requests identified, Budget Shaping determines how many candidate tokens each request submits for target verification. Target-model verification transitions from memory-bound to compute-bound as the number of candidate tokens grows. Below a hardware-determined inflection point, verifying additional tokens introduces minimal latency overhead compared with standard batched decoding; beyond it, latency grows approximately linearly. SpecRhythm therefore uses a roofline-guided verification budget, denoted as $B_{\text{roof}}(t)$, to bound the total number of candidate tokens verified in cycle t . At runtime, the system obtains $B_{\text{roof}}(t)$ from an offline profiling table indexed by the target model, the active batch size, and the context-length range. SpecRhythm keeps the total number of assigned candidates within this budget, ensuring target verification remains within the profiled low-latency region.

Within $B_{\text{roof}}(t)$, SpecRhythm shapes each request's speculative tree using both SLO urgency and expected acceptance benefit. For each request, the draft model first generates an exploratory set of *draft tokens*. SpecRhythm then evaluates these tokens using draft-model confidence estimates and selects a refined subset to construct the final candidate tree submitted to the target model. We refer to these selected tokens as *candidate tokens*. A higher-confidence candidate is more likely to be included in the accepted prefix, whereas deeper or lower-confidence candidates are more likely to be rejected. Thus, each candidate token consumes one verification slot but contributes a different expected value. SpecRhythm preserves prefix dependencies during candidate token selection, ensuring that a token can be submitted for target verification only if all draft tokens it depends on are also included.

SpecRhythm applies a two-step allocation policy. First, it allocates candidates to requests with projected progress gaps, prioritizing high-confidence tokens that help cover $a_i^{\text{need}}(t)$. This step prevents tight SLO requests from missing their TPOT targets by waiting until the normal batch rotation completes. To avoid wasting the global budget on low-value speculation, each request is capped by a per-request maximum budget, and candidate allocation is ranked according to estimated acceptance benefit. In the second step, any remaining candidate slots are assigned to high-confidence candidates across all requests, weighted by residual urgency. This improves residual goodput benefit without ignoring SLO urgency.

Budget shaping and eager continuation are coupled. Rolling Eager Continuation provides an ahead-of-turn drafting opportunity, while budget shaping determines the depth and width of that opportunity. An urgent request with high draft

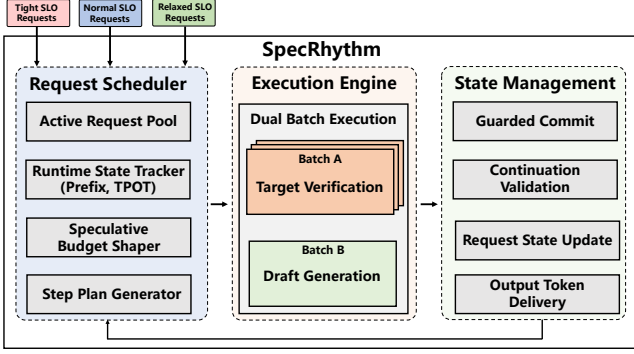


Figure 6. SpecRhythm system architecture.

confidence receives a larger speculative budget; as its projected progress gap closes or its expected acceptance benefit drops, the assigned budget contracts. In this way, eager continuation is not a binary fast path, but a controlled execution opportunity governed by Individual Speculative Budget Shaping.

5 Implementation

5.1 System Overview

SpecRhythm is structured around three logical stages: request scheduling, concurrent execution, and state management. Figure 6 illustrates how these stages interact within a service step. The request scheduler ingests heterogeneous traffic with diverse TPOT targets into an active request pool. It maintains the runtime state of each active request through a state tracker, recording delivered output tokens, accumulated decoding time, TPOT targets, recent acceptance behavior, and eager-continuation metadata. At the beginning of each step, the scheduler estimates the waiting time and projected progress gap for all active requests. The budget shaping module then determines the request-level speculative budget under the current execution constraints. Finally, the scheduler produces a step-level execution plan detailing which batch enters verification, which batch enters drafting, which requests receive eager continuations, and the exact speculative budget assigned to each selected request.

The execution engine follows this plan to perform dual-batch execution. It runs target verification for the active verification batch while concurrently generating drafts for the drafting batch. If the plan includes eager requests, the draft worker generates their continuation draft tokens within the same drafting window, avoiding the creation of an additional execution stage. After execution completes, the state management module closes the control loop. It performs guarded commits for verified tokens, validates eager continuations, updates request states, and delivers verified output tokens. The updated states are subsequently fed back to the request scheduler for the next service step. This separation

keeps SLO-aware scheduling decisions within the control plane, allowing the execution engine to focus exclusively on dispatching the planned GPU workloads.

5.2 Runtime Execution Loop

SpecRhythm consolidates the decisions of each service step into a single execution plan. The execution engine submits GPU workloads strictly according to this plan, enabling coordinated execution of draft and verification without requiring fine-grained synchronization between the draft and target workers during the step. At runtime, SpecRhythm maintains active requests in two logical execution slots. Completed requests are removed after a guarded commit, newly admitted requests are inserted into the less-loaded slot, and active requests retain their assigned slot as long as they have a pending draft or verification state. This request-boundary maintenance keeps the two slots dynamically balanced without invalidating the in-flight speculative state.

Since rolling eager continuation generates future tokens before the current speculative prefix is fully verified, SpecRhythm must preserve target-model correctness. The system attaches lightweight metadata to every eager continuation, recording the exact draft token sequence on which the continuation depends. During state management, the system compares the newly accepted prefix against this recorded dependency. If the target model accepts the entire dependent draft sequence, the eager continuation remains valid and is immediately promoted to a ready proposal for the subsequent verification step. Conversely, if any token in the dependent draft sequence is rejected, the eager continuation is discarded entirely, preventing tokens generated from an invalid prefix from being committed. After the guarded commit, the state management module updates the delivered token count, accumulated latency, and acceptance statistics. Promoted continuations allow urgent requests to bypass an additional normal drafting turn. Requests with discarded continuations simply fall back to the standard batch rotation and initiate a normal draft from the correctly verified prefix in a subsequent step.

5.3 System Profiling and Overhead Mitigation

SpecRhythm obtains the verification capacity $B_{\text{roof}}(t)$ from an offline profiling table. The profiler sweeps over the total candidate-token count across different target models, active batch sizes, and context-length ranges, and records the largest candidate budget for which verification latency remains within ϵ of standard batched decoding. At runtime, the scheduler looks up the corresponding $B_{\text{roof}}(t)$ for the current execution state and uses the two-step allocation policy to distribute this budget among normal drafting requests and eager continuation candidates.

To improve the efficiency of the draft worker, SpecRhythm merges standard drafts and eager continuations into a unified batched draft workload. For target verification, the target

worker flattens draft trees into a batched input using position IDs and attention masks that preserve tree dependencies. After the forward pass, the system maps the output logits back to their corresponding requests to perform verification. This batching strategy makes verification latency primarily depend on the total candidate-token count, while reducing per-request launch overhead.

The SpecRhythm scheduler operates on the CPU. Its core operations, including progress-gap estimation, eager-priority computation, and two-step budget allocation, are linear in the number of active requests. This lightweight design keeps scheduling overhead negligible compared to verification latency. To prevent eager continuations from consuming excessive memory and draft capacity, SpecRhythm caps both the depth and the total number of unverified proposals per request. If a request’s projected progress gap closes, or if the remaining draft window or verification budget becomes insufficient, the scheduler stops allocating eager budget to that request. This dynamic capping keeps metadata tracking lightweight and prevents eager drafting from crowding out standard drafting under heavy traffic.

6 Evaluation

6.1 Experimental Setup

Implementation. We implement SpecRhythm on top of nano-vLLM, extending its decoding scheduler with dual-batch execution, rolling eager continuation, and individual speculative budget shaping. All experiments are conducted on a single server equipped with 4 NVIDIA A100 80GB GPUs connected via NVLink and an Intel Xeon Platinum 8374C CPU @ 2.70 GHz.

Models. We evaluate SpecRhythm and the baselines using Llama3.1-70B-Instruct [32] and Qwen3-32B [33] as target models. Each is paired with a smaller draft model from the same model family: Llama3.2-1B-Instruct and Qwen3-0.6B, respectively. We do not apply task-specific distillation or fine-tuning to the draft models. All systems use the same decoding configuration, including maximum generation length, stopping criteria, and sampling parameters.

Baselines. We compare SpecRhythm with three speculative decoding baselines. **SpecInfer** [19] is a serial speculative decoding system that constructs and verifies speculative token trees. **AdaServe** [17] is a serial SLO-aware speculative serving system that assigns different speculative budgets according to latency targets. **PEARL** [18] is an asynchronous parallel speculative decoding system that overlaps drafting with verification through predrafting. For SpecInfer and AdaServe, the sequential execution allows the target model to utilize all four GPUs during the verification stage. For PEARL and SpecRhythm, which execute drafting and verification concurrently, we use one GPU for drafting and three GPUs for parallel target verification. This keeps the total GPU budget fixed across systems while matching each

system’s execution model. All systems use the same request traces, model pairs, SLO targets, and admission policy.

Workloads. We evaluate all systems using heterogeneous online workloads that contain three request categories with different TPOT SLOs. The tight class represents latency-sensitive code completion tasks and uses prompts from HumanEval [6]. Following the latency-sensitive interactive LLM serving target in MLPerf v5.0 [21], we set its TPOT SLO to 40 ms per token [26]. The normal class represents interactive chatbot requests and uses prompts from Alpaca [30]. We set its TPOT SLO to 50 ms per token to maintain responsive token streaming. The relaxed class represents background generation tasks such as summarization and uses prompts from CNN/DailyMail [2]. We set its TPOT SLO to 150 ms per token. The default workload uses a 6:2:2 mixture of tight, normal, and relaxed requests, which stresses the system with a high fraction of latency-sensitive traffic. We generate online arrivals by replaying timestamps from a production trace and rescaling the trace to obtain different average request rates. For each arriving request, we first sample its category according to the configured mixture and then uniformly sample a prompt from the corresponding dataset.

Metrics. Following the prefill-decode disaggregation methodology used in Mooncake [25], our evaluation focuses on the decoding stage. We model prefill as being served by a separate prefill pool, so a request enters our scheduler after its prompt has been processed and its KV cache is available to the decoding workers. We compute $TPOT_i = T_i^{\text{decode}} / N_i^{\text{out}}$, where T_i^{decode} measures only decoding time and N_i^{out} is the number of generated tokens. A request satisfies its SLO if $TPOT_i \leq \tau_i$. Goodput is the number of output tokens generated by SLO-satisfying requests divided by the decoding-stage measurement duration, and SLO attainment is the fraction of completed requests whose decoding-stage TPOT satisfies their assigned target.

6.2 Overall Performance

Changing active batch size. We first evaluate overall decoding performance as active batch pressure increases. We vary the active batch size from 8 to 64 and measure goodput and SLO attainment on both Llama3.1-70B-Instruct and Qwen3-32B. This experiment stresses whether each system can convert larger batch execution into timely request progress under heterogeneous TPOT SLOs.

Figure 7(a) and Figure 8(a) report goodput and SLO attainment on Llama3.1-70B-Instruct. At small batch sizes, AdaServe achieves slightly higher goodput and attainment than SpecRhythm, because serial speculation can avoid the overhead of maintaining overlapped execution when batch pressure is low. However, as the active batch size increases, SpecRhythm becomes increasingly advantageous. At batch size 64, SpecRhythm achieves 721.3 tokens/s goodput, outperforming SpecInfer, AdaServe, and PEARL by 3.75×, 1.27×,

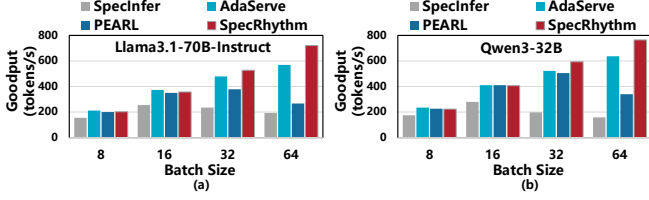


Figure 7. Goodput as active batch size increases. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. SpecRhythm achieves higher goodput under large batch pressure.

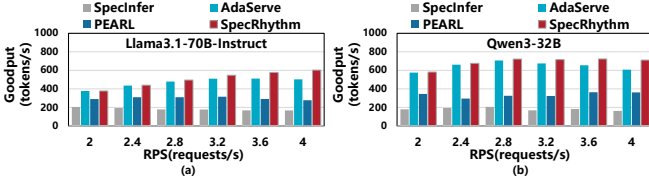


Figure 9. Goodput under increasing request arrival rate. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. SpecRhythm sustains higher goodput as online request pressure increases.

and 2.70 $\times$, respectively. It also reaches 80.7% SLO attainment, compared with 22.7% for SpecInfer, 61.4% for AdaServe, and 30.5% for PEARL.

Figure 7(b) and Figure 8(b) show similar trends on Qwen3-32B. At a batch size of 64, SpecRhythm achieves 764.8 tokens/s goodput, improving over SpecInfer, AdaServe, and PEARL by factors of 4.82 $\times$, 1.20 $\times$, and 2.25 $\times$, respectively. Its SLO attainment remains at 76.4%, while SpecInfer, AdaServe, and PEARL drop to 16.7%, 63.7%, and 28.8%, respectively.

These results show that SpecRhythm is more robust under large active batches. SpecInfer and AdaServe follow serial speculative execution, in which increasing speculative progress either lengthens iteration latency or requires reducing per-request budgets. PEARL overlaps drafting and verification, but its hidden drafting capacity is not allocated according to the runtime SLO urgency and can be wasted by invalidated drafts. SpecRhythm combines stable batch-level overlap with urgency-aware continuation and individual budget shaping, thereby preserving timely progress for more requests under high concurrency.

Changing request arrival rate. We next evaluate decoding performance under increasing online request pressure. We vary the average request arrival rate from 2 to 4 RPS while keeping the workload mixture and SLO targets fixed. This experiment evaluates whether each system can sustain timely progress as more requests enter the serving queue.

Figure 9(a) reports goodput on Llama3.1-70B-Instruct. As RPS increases, SpecInfer quickly saturates and its goodput decreases from 204.3 tokens/s to 167.3 tokens/s. PEARL also remains limited, reaching only 277.5 tokens/s at 4 RPS. AdaServe

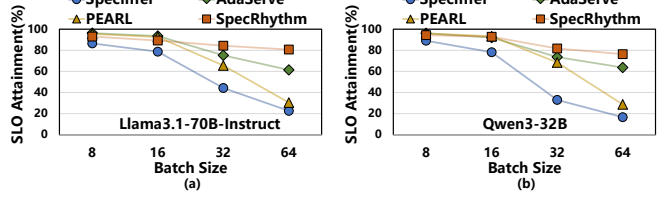


Figure 8. SLO attainment as active batch size increases. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. SpecRhythm preserves higher attainment as batch pressure grows.

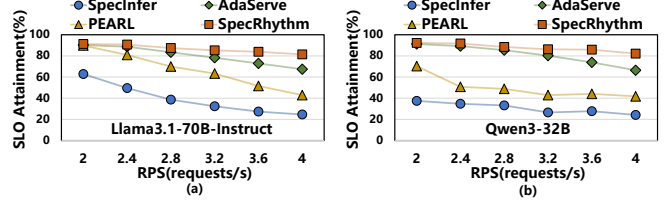


Figure 10. SLO attainment under increasing request arrival rate. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. SpecRhythm preserves higher attainment under heavier online traffic.

improves goodput at moderate load but stops scaling beyond 3.2 RPS. In contrast, SpecRhythm continues to improve goodput across the full range and reaches 603.3 tokens/s at 4 RPS, outperforming SpecInfer, AdaServe, and PEARL by 3.61 $\times$, 1.20 $\times$, and 2.17 $\times$, respectively. Figure 9(b) shows the same experiment on Qwen3-32B. SpecRhythm achieves the highest goodput across all RPS settings and reaches 711.0 tokens/s at 4 RPS. Compared with SpecInfer, AdaServe, and PEARL, this corresponds to 4.40 $\times$, 1.17 $\times$, and 1.97 $\times$ higher goodput, respectively. Although AdaServe remains competitive at moderate load, its goodput declines after 2.8 RPS, while SpecRhythm maintains higher goodput under heavier traffic.

Figure 10(a) and Figure 10(b) further report SLO attainment. On Llama3.1-70B-Instruct, SpecRhythm maintains 81.4% attainment at 4 RPS, compared with 24.6% for SpecInfer, 67.4% for AdaServe, and 43.0% for PEARL. On Qwen3-32B, SpecRhythm reaches 82.2% attainment at 4 RPS, compared with 24.2%, 66.4%, and 41.8% for the three baselines. These results show that SpecRhythm improves goodput by preserving timely request progress, rather than merely increasing token production.

Changing SLO scale. We further evaluate sensitivity to SLO strictness by scaling the TPOT targets of all request classes. A smaller scale corresponds to stricter latency requirements, while a larger scale allows more relaxed TPOT targets for requests. Figure 11(a) and Figure 11(b) show goodput on Llama3.1-70B-Instruct and Qwen3-32B, respectively. SpecRhythm consistently achieves the highest goodput across all SLO scales. In the strictest setting with a scale of 0.6, SpecRhythm reaches 422.7 tokens/s on Llama and

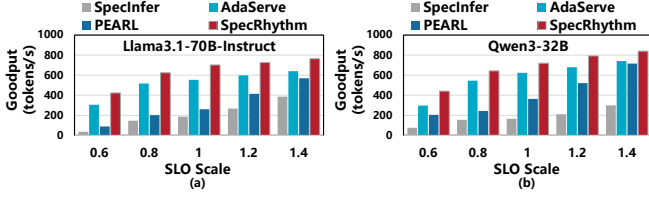


Figure 11. Goodput under varying SLO scale. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. Smaller scales indicate stricter TPOT targets. SpecRhythm maintains the highest goodput across all SLO settings.

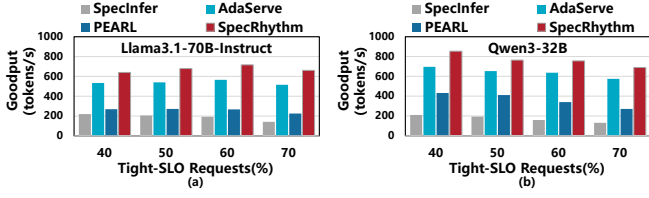


Figure 13. Goodput under varying tight request ratio. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. The remaining requests are split evenly between normal and relaxed classes.

440.9 tokens/s on Qwen, outperforming AdaServe by factors of 1.38 $\times$ and 1.48 $\times$, respectively. As SLOs become more relaxed, all systems improve, but SpecRhythm continues to maintain the best goodput.

Figure 12(a) and Figure 12(b) report SLO attainment. Under the strict scale of 0.6, SpecRhythm achieves 47.4% attainment on Llama and 44.4% on Qwen, while AdaServe achieves 33.2% and 29.0%, and PEARL achieves 9.2% and 19.4%. When the scale increases to 1.4, SpecRhythm reaches 92.5% and 93.5% attainment on the two models. These results show that SpecRhythm adapts more effectively to both strict and relaxed SLO regimes by prioritizing urgent requests and avoiding low-value speculative work.

Changing tight request ratio. We also vary the workload composition by increasing the fraction of tight requests from 40% to 70%, while splitting the remaining traffic evenly between normal and relaxed requests. This experiment evaluates whether each system can handle workloads with a higher fraction of latency-sensitive requests.

Figure 13(a) and Figure 13(b) report goodput on Llama3.1-70B-Instruct and Qwen3-32B, respectively. SpecRhythm achieves the highest goodput across all workload mixes. When the tight request ratio reaches 70%, SpecRhythm achieves 660.9 tokens/s on Llama and 689.8 tokens/s on Qwen, outperforming AdaServe by factors of 1.28 $\times$ and 1.20 $\times$, respectively. The advantage over SpecInfer and PEARL is larger because they either expose serial iteration overhead or fail to allocate hidden drafting capacity according to SLO urgency.

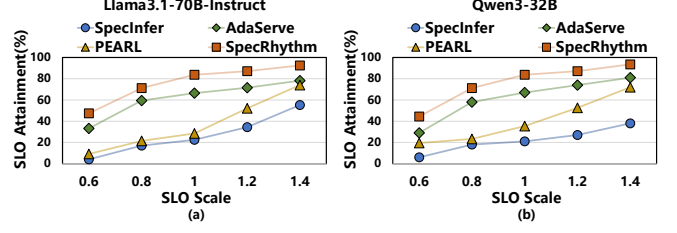


Figure 12. SLO attainment under varying SLO scale. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. SpecRhythm remains more robust as TPOT targets become stricter.

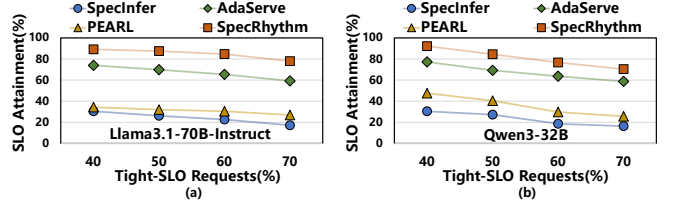


Figure 14. SLO attainment under varying tight request ratio. (a) Llama3.1-70B-Instruct. (b) Qwen3-32B. SpecRhythm preserves higher attainment as latency-sensitive traffic increases.

Figure 14(a) and Figure 14(b) show the corresponding SLO attainment. As the tight request ratio increases, all systems experience lower attainment, but SpecRhythm degrades more gracefully. At 70% tight requests, SpecRhythm maintains 78.0% attainment on Llama and 70.4% on Qwen, compared with 59.2% and 58.8% for AdaServe, and 27.0% and 25.7% for PEARL. These results show that SpecRhythm remains effective when latency-sensitive requests dominate the workload, because it gives urgent requests earlier drafting opportunities and shapes speculative budgets toward requests with higher expected goodput benefit.

6.3 Ablation Study

We conduct a cumulative ablation study to quantify the contribution of each component in SpecRhythm. Starting from vanilla speculative decoding, we progressively add the Dual-Batch Execution Pipeline, Rolling Eager Continuation, and Individual Speculative Budget Shaping. We run the ablation under the default heterogeneous online workload with an average request arrival rate of 3.6 RPS. Figure 15 reports goodput and SLO attainment on both Llama3.1-70B-Instruct and Qwen3-32B.

The results show that the three mechanisms are complementary across both target models. On Llama3.1-70B-Instruct, adding only the Dual-Batch Execution Pipeline reduces goodput from 182.4 to 150.4 tokens/s and SLO attainment from 27.7% to 22.8%. On Qwen3-32B, the same variant

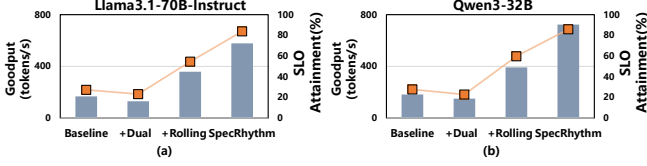


Figure 15. Cumulative ablation of SpecRhythm components on Llama3.1-70B-Instruct and Qwen3-32B. Bars report goodput, and orange squares report SLO attainment.

reduces goodput from 168.0 to 130.6 tokens/s and SLO attainment from 27.3% to 23.2%. This drop occurs because the dual-batch pipeline only changes the execution schedule; without rolling eager continuation and budget shaping, the hidden drafting window is not necessarily used by requests that need timely progress. Moreover, under dual-batch rotation, a request may wait for the complementary batch to finish verification before its own next verification turn, increasing latency for urgent requests unless the system actively uses the overlap window to advance them. Adding Rolling Eager Continuation converts the overlap window into timely progress. Goodput rises to 392.5 tokens/s with 59.8% attainment on Llama3.1-70B-Instruct, and to 358.5 tokens/s with 54.6% attainment on Qwen3-32B. Finally, enabling Individual Speculative Budget Shaping further improves goodput to 723.0 tokens/s with 85.9% attainment on Llama3.1-70B-Instruct, and to 577.8 tokens/s with 83.9% attainment on Qwen3-32B. These results show that SpecRhythm’s gains come from the combination of stable batch-level overlap, rolling eager continuation, and individual budget shaping.

7 Related Work

SLO-Aware LLM Serving. LLM serving systems increasingly optimize for latency-aware service quality rather than throughput alone. Orca [40] introduces iteration-level scheduling and selective batching, while vLLM [14] improves continuous batching through PagedAttention-based KV cache management. Sarathi-Serve [1] reduces prefill-decoding interference through chunked prefill, and DistServe [46] and Splitwise [24] disaggregate prefill and decoding to allocate resources based on their distinct latency and compute characteristics. Other systems further improve SLO attainment through preemptive scheduling [39], instance migration [28], or multiplexing across models and programs [8, 45]. These systems mainly control when requests are admitted, batched, migrated, or preempted. They improve queueing behavior and resource utilization, but they do not directly control how many output tokens each request can advance in one decoding step. Under heterogeneous TPOT SLOs, this distinction is important: requests with tight latency targets may need faster per-step progress, while relaxed requests can tolerate slower progress. SpecRhythm is complementary to these serving systems. Instead of scheduling requests only around

the decoding process, it uses speculative decoding to expose request-level progress as a controllable resource and allocates it according to SLO urgency and expected acceptance.

Speculative Decoding Systems. Speculative decoding and speculative sampling [5, 15] formalize the lossless draft-then-verify paradigm, and SpecInfer [19] extends it with token tree verification. Subsequent methods improve draft token generation through multihead prediction [4], feature-level drafting [16], hardware-aware tree construction [7], or self-speculative and early exit mechanisms [9, 41]. These works improve the efficiency or acceptance quality of speculative decoding, but they primarily target single-request acceleration or homogeneous serving settings. Recent systems explore execution level overlap between drafting and verification. PEARL [18] and SwiftSpec [44] reduce exposed drafting latency by overlapping or disaggregating the two stages, while Saguaro [13] further hides drafting overhead when predictions succeed. These systems improve the efficiency of speculative execution, but the hidden drafting work remains tied to acceptance outcomes and is not allocated according to the heterogeneous TPOT urgency.

AdaServe [17] is the closest to our work because it customizes speculative token trees for multi-SLO workloads. However, it still follows a serial execution structure, in which increasing the speculative budget directly increases the latency exposed at each iteration. SpecRhythm differs by separating the creation of drafting capacity from its allocation: a dual-batch execution pipeline provides a stable batch-level drafting window, while rolling eager continuation and individual budget shaping allocate that window to requests with high SLO urgency and high expected verified progress.

8 Conclusion

This paper presents SpecRhythm, a speculative LLM serving system that improves SLO-constrained goodput under heterogeneous TPOT targets. By decoupling the execution structure from speculative resource allocation, SpecRhythm addresses the latency-progress dilemma of serial decoding and the unstable overlap of parallel methods. At the execution layer, a dual-batch alternating pipeline creates a stable batch-level drafting window. At the control layer, rolling eager continuation and individual speculative budget shaping dynamically allocate this hidden drafting capacity according to runtime SLO urgency and draft confidence. Evaluations on heterogeneous online workloads show that SpecRhythm achieves up to $1.27\times$ higher goodput and $1.31\times$ higher SLO attainment over the strongest baseline, demonstrating the importance of treating speculative parallelism as an SLO-aware resource allocation problem rather than only a latency-hiding mechanism.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2024. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 3119–3137.
- [3] Marc Brysbaert. 2019. How many words do we read per minute? A review and meta-analysis of reading rate. *Journal of memory and language* 109 (2019), 104047.
- [4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 5209–5235.
- [5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023).
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [7] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yu-Hsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable and Robust Speculative Decoding. In *Proceedings of the 38th Annual Conference on Neural Information Processing Systems (NeurIPS 24)*.
- [8] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. 2024. MuxServe: flexible spatial-temporal multiplexing for multiple LLM serving. *arXiv preprint arXiv:2404.02015* (2024).
- [9] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 24)*. 12622–12642.
- [10] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, et al. 2024. DeepSpeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671* (2024).
- [11] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. 2025. Sola: Optimizing slo attainment for large language model serving with state-aware scheduling. *Proceedings of Machine Learning and Systems* 7 (2025).
- [12] Kaiyu Huang, Hao Wu, Zhuo Shi, Han Zou, Minchen Yu, and Qingjiang Shi. 2025. AdaSpec: Adaptive Speculative Decoding for Fast, SLO-Aware Large Language Model Serving. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 361–374.
- [13] Tanishq Kumar, Tri Dao, and Avner May. 2026. Speculative Speculative Decoding. In *Proceedings of the 14th International Conference on Learning Representations (ICLR 26)*.
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles (SOSP 23)*. 611–626.
- [15] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the International Conference on Machine Learning (ICML)*. 19274–19286.
- [16] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EA-GLUE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 28935–28948.
- [17] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Yi-Hsiang Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. 2026. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. In *Proceedings of the 21st European Conference on Computer Systems, EuroSys (EuroSys 26)*. 36–54.
- [18] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. 2025. PEARL: Parallel Speculative Decoding with Adaptive Draft Length. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 25)*.
- [19] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 24)*. 932–949.
- [20] MLCommons. 2024. Llama 2 70B: An MLPerf Inference Benchmark for Large Language Models.
- [21] MLCommons. 2025. MLPerf Inference v5.0 Advances Language Model Capabilities for GenAI.
- [22] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
- [23] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [24] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA 24)*. 118–132.
- [25] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.
- [26] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. 2020. Mlperf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 446–459.
- [27] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. 2024. Slora: Scalable serving of thousands of lora adapters. *Proceedings of Machine Learning and Systems* 6 (2024), 296–311.
- [28] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llmunix: Dynamic scheduling for large language model serving. In *Proceedings of the 18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 173–191.
- [29] Alexey Svyatkovskiy, Ying Zhao, Shengyu Fu, and Neel Sundaresan. 2019. Pythia: Ai-assisted code completion system. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2727–2735.

- [30] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Stanford alpaca: An instruction-following llama model.
- [31] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A Choquette-Choo, Jingyue Shen, Joe Kelley, et al. 2024. Codegemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409* (2024).
- [32] Llama Team. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [33] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [34] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, YaGuang Li, Hongrae Lee, Huaixiu Steven Zheng, Amin Ghafouri, Marcelo Menegali, Yanping Huang, Maxim Krikun, Dmitry Lepikhin, James Qin, Dehao Chen, Yuanzhong Xu, Zhifeng Chen, Adam Roberts, Maarten Bosma, Yanqi Zhou, Chung-Ching Chang, Igor Krivokon, Will Rusch, Marc Pickett, Kathleen S. Meier-Hellstern, Meredith Ringel Morris, Tulsee Doshi, Renelito Delos Santos, Toju Duke, Johnny Soraker, Ben Zevenbergen, Vinodkumar Prabhakaran, Mark Diaz, Ben Hutchinson, Kristen Olson, Alejandra Molina, Erin Hoffman-John, Josh Lee, Lora Aroyo, Ravi Rajakumar, Alena Butryna, Matthew Lamm, Viktoriya Kuzmina, Joe Fenton, Aaron Cohen, Rachel Bernstein, Ray Kurzweil, Blaise Agüera y Arcas, Claire Cui, Marian Croak, Ed H. Chi, and Quoc Le. 2022. LaMDA: Language Models for Dialog Applications. *arXiv preprint arXiv:2201.08239* (2022).
- [35] Dongsheng Wang, Natraj Raman, Mathieu Sibue, Zhiqiang Ma, Petr Babkin, Simerjot Kaur, Yulong Pei, Armineh Nourbakhsh, and Xiaomo Liu. 2024. Docllm: A layout-aware generative language model for multimodal document understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8529–8548.
- [36] Runze Wang, Qinggang Wang, Haifeng Liu, Long Zheng, Xiaofei Liao, Hai Jin, and Jingling Xue. 2026. Adaptive Draft Sequence Length: Enhancing Speculative Decoding Throughput on PIM-Enabled Systems. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1–15.
- [37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).
- [38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [39] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2026. FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 57–74.
- [40] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [41] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024. Draft& Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 24)*. 11263–11282.
- [42] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).
- [43] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. 2026. {JITServe}:{SLO-aware}{LLM} Serving with Imprecise Request Information. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 825–848.
- [44] Ziyi Zhang, Ziheng Jiang, Chengquan Jiang, Menghan Yu, Size Zheng, Haibin Lin, Xin Liu, and Henry Hoffmann. 2026. SwiftSpec: Disaggregated Speculative Decoding and Fused Kernels for Low-Latency LLM Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 26)*. 2197–2211.
- [45] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS 24)*.
- [46] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [47] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625* (2022).